

Worksheet 2

Student Name: HARSH

Branch: MCA

Semester: 3

Subject Name: BACKEND TECHNOLOGY LAB

UID: 24MCA20045

Section/Group: 1-B

Date of Performance: 15/08/2025

Subject Code: 24CAP-709

Aim/Overview of the practical:

Q. Create a Signup page and store the inputted data by user in file

1. Code for experiment/practical:

Signup.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0"/>
  <title>Signup Page</title>
  <style>
    *{ box-sizing: border-box; margin: 0; padding: 0; }
    body{
      background: linear-gradient(90deg, rgba(238,174,202,1) 0%, rgba(148,187,233,1) 100%);
      font-family: "Nunito", serif;
    }
    .form-container{
      display: flex;
      flex-wrap: wrap;
      max-width: 700px;
      height: auto;
      overflow: hidden;
      border-radius: 10px;
      background-color: rgba(255, 255, 255, 0.2);
      backdrop-filter: blur(10px);
      margin: 40px auto;
    }
    .form { padding: 20px; width: 50%; }
    .form h1 {
      text-align: center;
      color: #1a73e8;
      font-size: 2.5rem;
    }
  </style>
</head>
```

```
.form input {
  display: block;
  width: 80%;
  margin: 10px 20px;
  padding-left: 5px;
  border: 2px solid #1a73e8;
  border-radius: 5px;
  font-size: 1.125rem;
}

.form label{
  display: block;
  margin-left: 10%;
  color: #1a73e8;
  font-weight: bold;
}

.password-wrap{
  width: 100%;
  display: flex;
  align-items: center;
}

#eye-icon {
  position: relative;
  cursor: pointer;
  top: 0;
  right: 50px;
  color: #1a73e8;
}

.form .term{ margin-left: -27%; }

.form button {
  display: block;
  width: 80%;
  margin: 20px 20px;
  padding: 8px 0;
  border: none;
  border-radius: 5px;
  background-color: #1a73e8;
  color: white;
  font-size: 1.125rem;
  cursor: pointer;
  transition: 0.3s ease;
}

.form button:hover {
  background-color: transparent;

  border: 1px solid #4285f4;
  color: #4285f4;
}
```



```
.image-container { width: 50%; }
.image-container img {
  margin-top: 20px;
  width: 100%;
  height: auto;
  object-fit: cover;
}
@media screen and (max-width:768px) {
  .form-container { flex-direction: column; }
  .form { width: 100%; }
  .image-container { width: 100%; }
}
.error { color: red; font-size: 0.9rem; margin-left: 10%; }
</style>
</head>
<body>
<div class="form-container">
  <div class="form">
    <h1>Sign Up</h1>
    <form id="signupForm" action="/signup" method="POST">
      <label for="name">Name</label>
      <input type="text" id="name" name="name" placeholder="Enter name" required>

      <label for="email">Email</label>
      <input type="email" id="email" name="email" placeholder="Enter your email" required>

      <label for="password">Password</label>
      <div class="password-wrap">
        <input type="password" id="password" name="password" placeholder="Enter your password" required>
        <i class="fa fa-eye" id="eye-icon" onclick="togglePassword()"></i>
      </div>

      <input type="checkbox" class="term" id="terms" name="terms" required>
      <label for="terms">I agree to the Terms and Conditions</label>

      <button type="submit">Signup</button>
    </form>
  </div>

  <div class="image-container">
    
  </div>
</div>

<!-- Inline JS for toggle -->
<script>
```

```
function togglePassword() {  
    const pass = document.getElementById("password");  
    if (pass.type === "password") {  
        pass.type = "text";  
    } else {  
        pass.type = "password";  
    }  
}  
  
</script>  
</body>  
</html>
```

Success.html:

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
    <meta charset="UTF-8">  
    <meta name="viewport" content="width=device-width, initial-scale=1.0">  
    <title>Signup Success</title>  
    <style>  
        body {  
            font-family: Arial, sans-serif;  
            text-align: center;  
            padding: 50px;  
            background: linear-gradient(135deg, #89f7fe, #66a6ff);  
        }  
        .box {  
            background: white;  
            padding: 30px;  
            border-radius: 12px;  
            display: inline-block;  
            box-shadow: 0 5px 15px rgba(0,0,0,0.2);  
        }  
        h1 { color: green; }  
        a {  
            display: inline-block;  
            margin-top: 20px;  
            padding: 10px 20px;  
            text-decoration: none;  
            background: #1a73e8;  
            color: white;  
            border-radius: 8px;  
        }  
        a:hover { background: #1558b0; }  
    </style>
```

```
</head>
<body>
  <div class="box">
    <h1>User added successfully!</h1>
    <a href="https://en.wikipedia.org/wiki/Express">Login</a>
  </div>
</body>
</html>
```

App.js:

```
const express = require('express');
const bodyParser = require('body-parser');
const mongoose = require('mongoose');
const bcrypt = require('bcrypt');
const path = require('path');

const app = express();
const PORT = 3000;

// Connect to MongoDB
mongoose.connect('mongodb://localhost:27017/signupApp', {
  useNewUrlParser: true,
  useUnifiedTopology: true,
})
.then(() => console.log('✅ MongoDB connected'))
.catch(err => console.error('❌ MongoDB connection error:', err));

// Define User Schema
const userSchema = new mongoose.Schema({
  name: { type: String, required: true, minlength: 3 },
  email: { type: String, required: true, unique: true, match: /^[^ ]+@[^ ]+\.[a-z]{2,}$/ },
  password: { type: String, required: true, minlength: 6 },
});

const User = mongoose.model('User', userSchema);

// Middleware
app.use(bodyParser.urlencoded({ extended: true }));
app.use(express.static(path.join(__dirname, 'public')));

// Default route
app.get('/', (req, res) => {
  res.sendFile(path.join(__dirname, 'public', 'signup.html'));
```

```
});
```

// Handle form submission

```
app.post('/signup', async (req, res) => {  
  const { name, email, password } = req.body;  
  
  try {  
    if (!name || name.trim().length < 3) {  
      return res.status(400).send('Name must be at least 3 characters long');  
    }  
    if (!email || !/^[^ ]+@[^ ]+\.[a-z]{2,}$/.test(email)) {  
      return res.status(400).send('Invalid email format');  
    }  
  
    if (!password || password.length < 6) {  
      return res.status(400).send('Password must be at least 6 characters long');  
    }  
  
    // Check if email already exists  
    const existingUser = await User.findOne({ email });  
    if (existingUser) {  
      return res.status(400).send('Email already registered');  
    }  
  
    // Encrypt password  
    const hashedPassword = await bcrypt.hash(password, 10);  
  
    // Save user in DB  
    const newUser = new User({ name, email, password: hashedPassword });  
    await newUser.save();  
  
    res.redirect('/success.html'); //  Redirect on success  
  
  } catch (err) {  
    console.error('Error during signup:', err);  
    res.status(500).send('Internal Server Error');  
  }  
});  
  
// Start server  
app.listen(PORT, () => {  
  console.log( Server is running on http://localhost:${PORT});  
});
```

OUTPUT:

```
PS C:\vscode> cd signup-app
PS C:\vscode\signup-app> node app.js
(node:18564) [MONGODB DRIVER] Warning: useNewUrlParser is a deprecated option: useNewUrlParser has no effect since Node.js Driver version 4.0.0
and will be removed in the next major version
(Use `node --trace-warnings ...` to show where the warning was created)
(node:18564) [MONGODB DRIVER] Warning: useUnifiedTopology is a deprecated option: useUnifiedTopology has no effect since Node.js Driver version
4.0.0 and will be removed in the next major version
✓ Server is running on http://localhost:3000
✓ MongoDB connected
```

Sign Up

Name

Email

Password



I agree to the Terms and Conditions

Signup



User added successfully!

Login

```
_id: ObjectId('68aad5afea10cb0d958057de')
name : "Praveen Kumar "
email : "praveenkumar31102005@gmail.com"
password : "$2b$10$j3okcktmJvFuI7JrGmzy6.b1TKJWJQPULTmqIYwBHdnCqvdIsAxQu"
__v : 0
```

```
_id: ObjectId('68adacaaa550499c941ce49b')
name : "Harsh"
email : "harsh@gmail.com"
password : "$2b$10$RM8FZXM5AmvzWHT6y7iFWeIJGSruZttlo.xeovfIux3blNf3k8gea"
__v : 0
```

Learning Outcomes:

- Learned to set up a Node.js server using Express.
- Connected and interacted with MongoDB using Mongoose.
- Handled form submissions with body-parser.
- Validated user input for name, email, and password.
- Implemented password hashing using bcrypt.
- Created a beautiful frontend success page with HTML/CSS.
- Handled errors like duplicate emails and invalid inputs.

Evaluation Grid:

Sr. No.	Parameters	Marks Obtained	Maximum Marks
1.	Demonstration and Performance		12
2.	Worksheet		8
3.	Via		10